Exercices en C — Pointeurs et allocation dynamique

Exercice 1 – Fonction de swap via pointeurs

Écrire un programme en C qui :

1. Demande à l'utilisateur de saisir deux entiers a et b.
2. Implémente une fonction :

   ```
   void swap(int *x, int *y);
   ```

   qui échange les valeurs pointées par x et y à l'aide d'une variable temporaire.
3. Dans le `main`, appelle `swap(&a, &b)` puis affiche les nouvelles valeurs de a et b.

Contraintes : - L'échange doit se faire exclusivement avec les pointeurs. - Aucun `return` d'une valeur (tout passe par les adresses).

Exercice 2 – Somme et moyenne d'un tableau via pointeurs

Écrire un programme en C qui :

1. Déclare un tableau d'entiers de taille 5, par exemple :

   ```
   int tab[5];
   ```

2. Demande à l'utilisateur de saisir les 5 valeurs du tableau.
3. Implémente une fonction :

   ```
   void calculer_somme_moyenne(int *t, int taille, int *somme, float
   ↪  *moyenne);
   ```

   qui :

   - parcourt le tableau en utilisant un pointeur ou l'arithmétique de pointeurs,
   - calcule la somme des éléments,
   - calcule la moyenne (type `float`),
   - renvoie les résultats via `somme` et `moyenne`.
4. Affiche la somme et la moyenne dans le `main`.

Contraintes : - Ne pas utiliser la notation `t[i]` dans la fonction.

Exercice 3 – Tableau de taille dynamique rempli par l'utilisateur

Écrire un programme en C qui :

1. Demande à l'utilisateur un entier n strictement positif : la taille du tableau.
2. Alloue dynamiquement un tableau de n entiers avec `malloc`.
3. Demande à l'utilisateur de saisir les n valeurs.
4. Affiche tous les éléments du tableau ainsi que la valeur minimale et la valeur maximale.
5. Libère la mémoire avant de quitter.

Contraintes : - Vérifier que `malloc` ne renvoie pas `NULL`. En cas d'erreur, afficher un message et terminer proprement.

Fonction suggérée :

```
void trouver_min_max(int *t, int taille, int *min, int *max);
```

Exercice 4 – Fonction qui renvoie un tableau dynamique

Écrire un programme en C qui contient la fonction suivante :

```
int* generer_tableau(int n, int valeur_depart);
```

Cette fonction doit : 1. Allouer dynamiquement un tableau de n entiers. 2. Remplir ce tableau avec des valeurs croissantes à partir de `valeur_depart`:- `tab[0] = valeur_depart`- `tab[1] = valeur_depart + 1` - etc. 3. Renvoyer le pointeur sur le tableau. 4. En cas d'erreur d'allocation, renvoyer `NULL`.

Dans le `main` :

1. Demander à l'utilisateur n (taille du tableau, > 0) et `valeur_depart`.
2. Appeler `generer_tableau(n, valeur_depart)`.
3. Si le pointeur renvoyé est non nul, afficher les n valeurs générées.
4. Libérer la mémoire allouée avec `free`.

Exercice 5 – Pointeur vers une structure et modification via fonction

On souhaite représenter un étudiant avec la structure suivante :

```
struct Etudiant {
    char nom[30];
    int age;
};
```

Écrire un programme en C qui :

1. Déclare une variable de type `struct Etudiant e;`.
2. Déclare un pointeur `struct Etudiant *pe;` et le fait pointer sur e.
3. Implémente une fonction :

   ```
   void initialiser_etudiant(struct Etudiant *et);
   ```

   qui :

   - demande à l'utilisateur de saisir le nom et l'âge de l'étudiant,
   - remplit les champs de la structure passée en paramètre via le pointeur (`et->nom`, `et->age`).

Dans le `main` :

1. Appeler `initialiser_etudiant(pe);`.
2. Afficher ensuite le nom et l'âge de l'étudiant en utilisant le pointeur pe.

Contraintes : - L'accès aux membres dans la fonction doit se faire avec l'opérateur `->` via le pointeur.

Exercice 6 – Tableau dynamique de structures

On reprend la structure `Etudiant` de l'exercice précédent :

```
struct Etudiant {
    char nom[30];
    int age;
};
```

Écrire un programme en C qui :

1. Demande à l'utilisateur combien d'étudiants il veut saisir (n, > 0).
2. Alloue dynamiquement un tableau de n structures :

   ```
   struct Etudiant *classe = malloc(n * sizeof(struct Etudiant));
   ```

3. Pour chaque étudiant :

   - demander le nom et l'âge,
   - stocker ces informations dans le tableau dynamique.

4. À la fin de la saisie, afficher :

- la liste de tous les étudiants (nom + âge),
- l'étudiant le plus âgé (nom + âge).

5. Libérer la mémoire allouée.

Contraintes : - Vérifier le retour de `malloc`. - Le calcul de l'étudiant le plus âgé doit être fait dans une fonction séparée qui prend le tableau et sa taille en paramètre.

Exercice 7 – Chaîne de caractères dynamique et réallocation

Écrire un programme en C qui :

1. Alloue dynamiquement une chaîne de caractères de taille 11 (10 caractères + le `\0`) :

   ```
   char *chaine = malloc(11 * sizeof(char));
   ```

2. Vérifie que l'allocation a réussi.
3. Demande à l'utilisateur de saisir un mot ou une petite phrase de longueur maximale 10 caractères, et le stocke dans `chaine`.
4. Affiche la chaîne et sa longueur (calculée manuellement, sans `strlen`).
5. Demande ensuite à l'utilisateur de saisir une nouvelle chaîne, cette fois potentiellement plus longue (par exemple jusqu'à 30 caractères).
6. Réalloue la mémoire avec `realloc` pour pouvoir stocker cette nouvelle chaîne (taille suffisante + `\0`).
7. Stocke la nouvelle chaîne dans la zone réallouée, puis l'affiche.
8. Libère la mémoire allouée.

Contraintes : - Vérifier que `realloc` ne renvoie pas `NULL`. En cas d'échec, ne pas perdre l'ancienne adresse. - Bien gérer le caractère de fin de chaîne `\0`.

Exercice 8 – Mini linked list (liste chaînée simple)

On souhaite manipuler une liste chaînée d'entiers.

1. Définir la structure :

   ```
   struct Noeud {
       int valeur;
       struct Noeud *suivant;
   };
   ```

2. Écrire une fonction qui ajoute un nouvel élément en tête de liste :

   ```
   struct Noeud* ajouter_en_tete(struct Noeud *tete, int valeur);
   ```

   Cette fonction doit :

   - allouer dynamiquement un nouveau nœud,
   - initialiser son champ `valeur`,
   - faire pointer son champ `suivant` vers l'ancienne tête,
   - renvoyer la nouvelle tête.

3. Écrire une fonction d'affichage :

   ```
   void afficher_liste(struct Noeud *tete);
   ```

   qui parcourt la liste du début à la fin et affiche les valeurs.

4. Dans le `main` :

   - créer une liste vide : `struct Noeud *tete = NULL;`
   - demander à l'utilisateur de saisir des entiers positifs ; pour chaque entier saisi (différent de 0), appeler `ajouter_en_tete` pour l'insérer dans la liste,
   - arrêter la saisie lorsque l'utilisateur entre `0`,
   - afficher la liste avec `afficher_liste(tete);`
   - libérer tous les nœuds de la liste avec `free`.